

Reviewer Pack — Minimal Number of Brauer Classes and Communication Rank Grow...

Entry b4757a993bbc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 03:08:40 UTC

Minimal Number of Brauer Classes and Communication Rank Growth

Entry ID: b4757a993bbc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 03:08:40 UTC

1. Conjecture

Field A (mathematical branch): Brauer Theory **Field B** (complexity object): Communication Complexity

Statement:

For any given k -colorable graph G , the minimal number of Brauer classes ($\text{br}(G)$) associated with its edge set is linearly correlated with the growth rate of its communication rank, such that $\text{br}(G) = \Theta(\text{growth_rate}(\text{communication_rank}(G)))$.

Rationale (proposer's reasoning):

Brauer theory, which studies the fundamental properties of vector spaces over finite fields, has not been extensively applied to communication complexity. This conjecture suggests a potential link between algebraic properties of edge sets and the computational difficulty of distributing information, which could expose new structural insights into the hardness of distribution problems.

Taxonomy category: Brauer Theory Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3e89d29e673fd71f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if there exists a linear correlation with a Pearson's r -value ≥ 0.7 between the minimal number of Brauer classes ($\text{br}(G)$) for each graph G and its communication rank growth rate, measured across at least 30 independent seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Brauer Theory' AND 'communication complexity' AND 'graph theory' - 'edge set Brauer classes' AND 'communication complexity' AND 'growth rate' - 'minimum number of Brauer classes' AND 'communication rank growth' AND 'graph properties'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_colorable_graph(n, k):
        if n <= 1 or k < 2:
            return []

        graph = [[] for _ in range(n)]
        colors = list(range(1, k + 1))
        color_count = [0] * (k + 1)

        for i in range(n):
            available_colors = [c for c in colors if color_count[c] < n // k]
            if not available_colors:
                break
            color = random.choice(available_colors)
            graph[i].append(color)
            color_count[color] += 1

        return graph

    def compute_brauer_classes(graph):
        n = len(graph)
        if n == 0:
            return 0

        # Construct the field over finite fields
        field_size = n + 1
        field_elements = [i for i in range(field_size)]
```

```

# Compute Brauer classes (simplified example)
brauer_classes = set()
for node in graph:
    if node:
        color = node[0]
        brauer_classes.add((color, node))

return len(brauer_classes)

def compute_communication_rank(graph):
    n = len(graph)
    if n == 0:
        return 0

# Simulate distributed algorithms (simplified example)
communication_rank = sum(len(node) for node in graph) / n
return communication_rank

results = []
for n in [5, 10, 15, 20, 30, 40]:
    graph = generate_k_colorable_graph(n, k=3)
    if not graph:
        continue

    br_value = compute_brauer_classes(graph)
    comm_rank = compute_communication_rank(graph)

    results.append({
        "n": n,
        "br_value": br_value,
        "comm_rank": comm_rank
    })

if len(results) < 30:
    return {
        "metric_name": "Pearson's r-value",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results) if results else 0,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

br_values = [result["br_value"] for result in results]
comm_ranks = [result["comm_rank"] for result in results]

mean_br = sum(br_values) / len(br_values)
mean_comm_rank = sum(comm_ranks) / len(comm_ranks)

covariance = sum((br - mean_br) * (comm - mean_comm_rank) for br, comm in zip(br_values, comm_ranks))
variance_br = sum((br - mean_br) ** 2 for br in br_values)
variance_comm_rank = sum((comm - mean_comm_rank) ** 2 for comm in comm_ranks)

if variance_br == 0 or variance_comm_rank == 0:
    return {

```

```

        "metric_name": "Pearson's r-value",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": False,
        "counterexample": "insufficient_variance"
    }

    pearsons_r = covariance / (math.sqrt(variance_br) * math.sqrt(variance_comm_rank))

    return {
        "metric_name": "Pearson's r-value",
        "metric_value": pearsons_r,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": abs(pearsons_r) >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results if res["metric_value"] is not None) /
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results if res["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59958fd0.py", line 127, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59958fd0.py", line 72, in run_trial
    br_value = compute_brauer_classes(graph)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59958fd0.py", line 53, in compute_brauer

```

```
brauer_classes.add((color, node))
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the linear correlation required to support the conjecture. | next: Review and debug the test code to ensure it can complete without crashing and produce the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14210
2	propose	ollama_remote	glm4:latest	0	0	12156
3	preregistration	ollama_remote	glm4:latest	0	0	9395
4	novelty	ollama_remote	glm4:latest	0	0	8422
5	novelty	ollama_remote	glm4:latest	0	0	10835
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18628
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18043
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	57972
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28433
10	judge	ollama_remote	glm4:latest	0	0	54483

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 232577 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/b4757a993bbc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b4757a993bbc.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b4757a993bbc.tar.gz (if generated)